0133. Clone Graph

ty: Medium | Category: Graphs | LeetCode: https://leetcode.com/problems/clone-

Problem

- Deep-copy a connected undirected graph given one node (each node has val and a neighbors list).

Intuition

- DFS through the graph, keeping a map original → clone.
- The map doubles as the visited set and handles cycles: if a node is already cloned, reuse the clone.

Brute Force

- No meaningfully slower approach; BFS with the same map is an equal alternative.

Optimal Approach

- Clone(node): if null return null; if in map return map.get(node).
- Copy = new Node(val); map.put(node, copy) before visiting neighbours (breaks cycles).
- For each neighbour: copy.neighbors.add(clone(nb)).

Worked Example

```
adjacency [[2,4],[1,3],[2,4],[1,3]] (a square).

clone 1 -> clone 2 -> clone 3 -> clone 4 -> neighbour 1 already in map (reuse)
back in 3: neighbour 4 done ; back in 2: neighbour 3 done
... all 4 nodes copied, edges point to copies
```

Java Solution

```java
public class CloneGraph {
    public static class Node {
        public int val;
        public List<Node> neighbors = new ArrayList<>();
        public Node(int val) { this.val = val; }
    }

    private final Map<Node, Node> map = new HashMap<>();

    public Node cloneGraph(Node node) {
        if (node == null) return null;
        Node existing = map.get(node);
        if (existing != null) return existing;
        Node copy = new Node(node.val);
        map.put(node, copy);
        for (Node nb : node.neighbors) copy.neighbors.add(cloneGraph(nb));
        return copy;
    }
}
```

Complexity

- Time: O(V + E). Space: O(V).

Pitfalls

- Insert into the map before recursing, or a cycle recurses forever.